

notebook_solution

January 12, 2026

```
[1]: import os
      # Ensure we are in the correct directory for data loading
      if os.path.basename(os.getcwd()) != "lab2_neural_models":
          if os.path.exists("./lab2_neural_models"):
              os.chdir("./lab2_neural_models")
      print(f"Current working directory: {os.getcwd()}")
```

Current working directory: /Users/mateu/Education/sem9/ml/ml_time_series_forecasting_course/lab2_neural_models

1 Neural forecasting models

We will cover neural models for time series forecasting, both trained from scratch and pretrained. We will use varied libraries, depending on the model, for example: - [sktime](#) - general time series processing - [neuralforecast](#) - a lot of neural models for time series, e.g. DLinear, N-BEATS - [PyTorch](#) - deep learning framework - [timesfm](#) - official TimesFM implementation (and loading pretrained model)

Use tutorials, quickstarts, GitHub pages etc. of those libraries as necessary.

```
[2]: import warnings

warnings.simplefilter(action="ignore", category=FutureWarning)
warnings.simplefilter(action="ignore", category=UserWarning)

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from neuralforecast import NeuralForecast
from neuralforecast.models import NBEATS, TSMixer
from neuralforecast.losses.pytorch import MAE
```

1.1 Datasets and evaluation

We will use 2 datasets: 1. [Italian pasta dataset](#), same as in the first notebook. 2. [Polish energy production data](#), as published by Energy Instrat and ENTSO-e, from data by PSE (Polskie Sieci Elektroenergetyczne).

Both are multivariate and focused on long-term forecasting.

1.1.1 Italian pasta

Data loading and visualization This dataset technically multivariate, but it has data from 4 different companies with very different characteristics, so it may have pretty weak cross-series dependencies. We will consider a simplified variant with no exogenous variables.

```
[3]: from sktime.utils.plotting import plot_series

df_pasta = pd.read_csv("italian_pasta.csv")
for num in [1, 2, 3, 4]:
    company_qty_cols = [col for col in df_pasta.columns if col.
        ↳startswith(f"QTY_B{num}")]
    df_pasta[f"value_B{num}"] = df_pasta[company_qty_cols].sum(axis="columns")

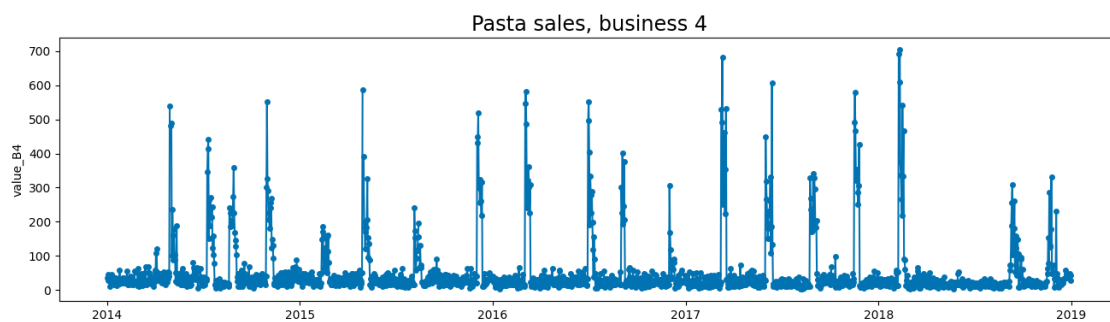
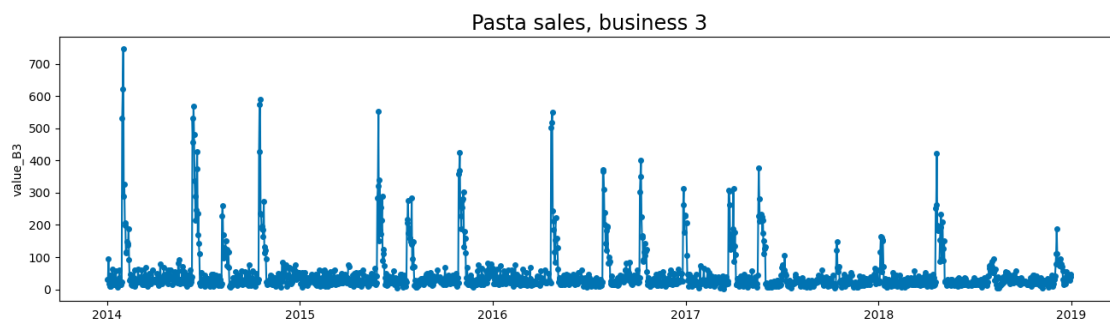
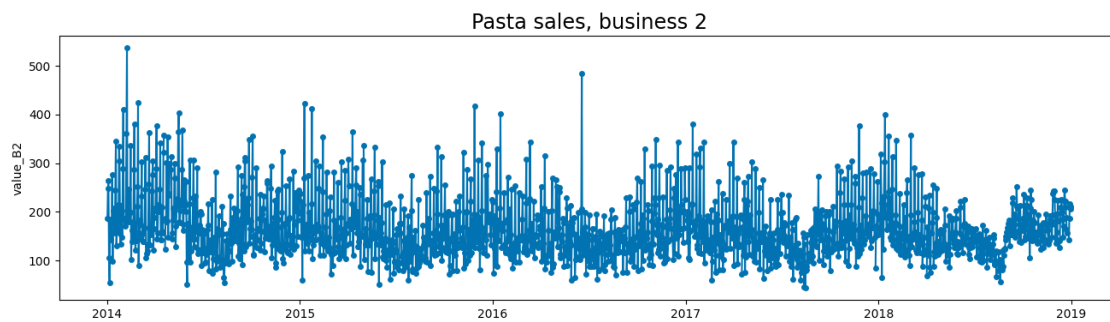
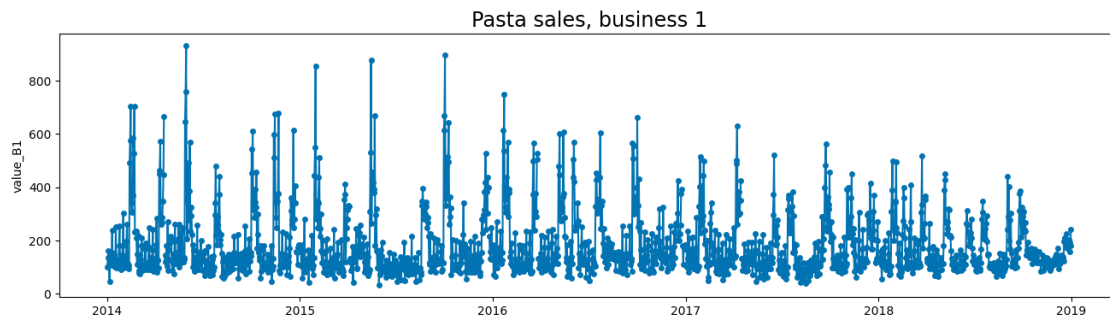
df_pasta = df_pasta.set_index(pd.to_datetime(df_pasta["DATE"])).asfreq("d")
df_pasta = df_pasta[["value_B1", "value_B2", "value_B3", "value_B4"]]
df_pasta
```

```
[3]:
```

	value_B1	value_B2	value_B3	value_B4
DATE				
2014-01-02	101.0	186.0	32.0	36.0
2014-01-03	136.0	248.0	32.0	27.0
2014-01-04	162.0	264.0	94.0	45.0
2014-01-05	106.0	106.0	18.0	29.0
2014-01-06	47.0	54.0	9.0	11.0
...	...	...	...	...
2018-12-27	203.0	143.0	30.0	39.0
2018-12-28	192.0	187.0	28.0	44.0
2018-12-29	158.0	217.0	44.0	48.0
2018-12-30	182.0	211.0	40.0	27.0
2018-12-31	243.0	206.0	48.0	42.0

[1825 rows x 4 columns]

```
[4]: for num in [1, 2, 3, 4]:
    plot_series(df_pasta[f"value_B{num}"], title=f"Pasta sales, business {num}")
```



Evaluation Similarly to the first notebook, we will be interested in long-term forecasting, predicting the daily sales for 2018, based on previous years. Since we have 4 time series with different scales, MASE is a great metric, since it can be averaged across series.

```
[5]: from sktime.transformations.series.impute import Imputer

df_pasta_train = df_pasta[df_pasta.index < "2018-01-01"]
df_pasta_test = df_pasta[df_pasta.index >= "2018-01-01"]

imputer = Imputer(method="ffill")
df_pasta_train = imputer.fit_transform(df_pasta_train)
df_pasta_test = imputer.transform(df_pasta_test)

print(f>Data size: train {len(df_pasta_train)}, test {len(df_pasta_test)}")
```

Data size: train 1460, test 365

1.1.2 Polish energy production

Data loading and visualization Energy mix is composed of multiple energy sources. It typically consists of multiple components: - slow-changing base, e.g. coal, nuclear - faster changing and controllable sources, e.g. gas, oil, hydro - very cheap, but uncontrollably changing renewables, e.g. wind, solar

The resulting production is always limited by the grid efficiency, which is very low in Poland, resulting in e.g. refusing to connect more prosumer solar installations. As such, the production limits are monitored and controlled, and cross-series dependencies are often quite strong.

We will aggregate the energy sources a bit, and consider: - coal (and derivatives) - hydro (from all sources) - solar - wind - all others, e.g. oil (petroleum), biomass

Since units are GWh (10^9 Wh, Whatt hours), values are very high, so we will consider thousands of GWh, i.e. TWh (10^{12} Wh). It is not a standard unit, but should help with numerical stability for methods that do not perform standardization or scaling.

Data from PSE has changed its format and processing at 13.06.2024, and values since this date are in 15-minutes intervals, compared to 1-hour from before. As such, we divide them by 4, to have the same unit.

If you want to know more about energy production and demand, see e.g. [this video](#) or [this video](#).

```
[6]: df_energy = pd.read_csv("electricity_production_entsoe_all.csv")
df_energy = df_energy.drop(columns="date_utc")
df_energy["date"] = pd.to_datetime(df_energy["date"], format="%d.%m.%Y %H:%M")
df_energy = df_energy.set_index("date")
df_energy = df_energy.resample("D").sum()

# aggregate energy sources
df_energy["coal"] = (
    df_energy["hard_coal"] + df_energy["coal-derived"] + df_energy["lignite"]
```

```

)
df_energy["hydro"] = (
    df_energy["hydro_pumped_storage"] +
    df_energy["hydro_run-of-river_and_poundage"] +
    df_energy["hydro_water_reservoir"]
)
df_energy["wind"] = df_energy["wind_onshore"]
df_energy["other"] = (
    df_energy["oil"] +
    df_energy["biomass"] +
    df_energy["other"] +
    df_energy["other_renewable"]
)
df_energy = df_energy[["coal", "gas", "hydro", "wind", "solar", "other"]]

# fix values and change units (GWh -> thousands of GWh)
df_energy[df_energy.index >= "13.06.2024"] /= 4
df_energy["other"][df_energy.index >= "13.06.2024"] /= 2
df_energy = df_energy / 1000

df_energy

```

```

[6]:

```

	coal	gas	hydro	wind	solar	other
date						
2015-01-02	298.90400	10.13800	9.25500	77.61100	0.0000	5.702000
2015-01-03	288.79200	10.25900	9.30400	80.98500	0.0000	5.155000
2015-01-04	271.66200	9.89600	7.83800	77.09600	0.0000	4.175000
2015-01-05	367.48300	10.03000	7.09000	43.30900	0.0000	5.627000
2015-01-06	343.57200	10.26700	7.06800	18.49900	0.0000	5.631000
...	...	...	...	...	...	...
2024-10-30	272.83625	68.80675	4.48175	87.23000	13.5885	16.192750
2024-10-31	252.18300	60.90650	4.63500	111.13200	20.7190	16.189625
2024-11-01	158.76150	43.92125	5.50675	169.88575	23.0975	14.869250
2024-11-02	190.79925	47.28800	5.18575	88.41450	45.9005	15.769750
2024-11-03	138.62925	38.76325	3.76750	71.03375	32.5945	11.860500

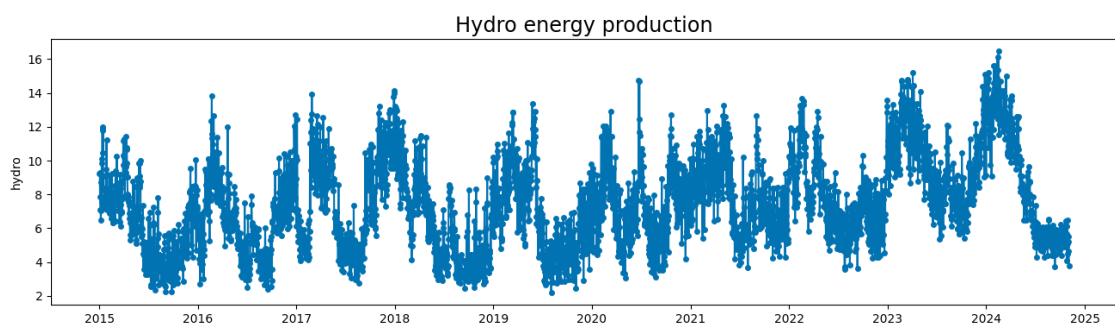
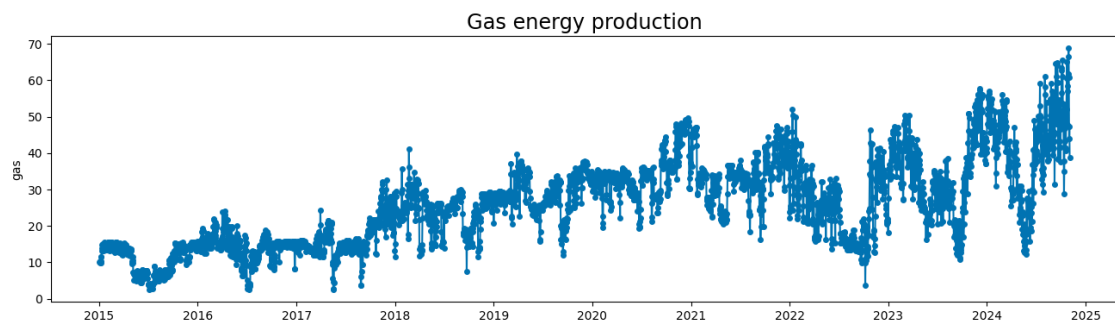
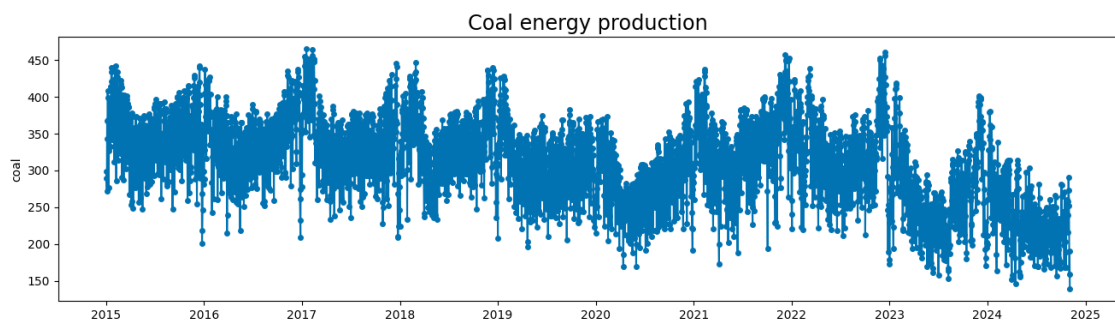
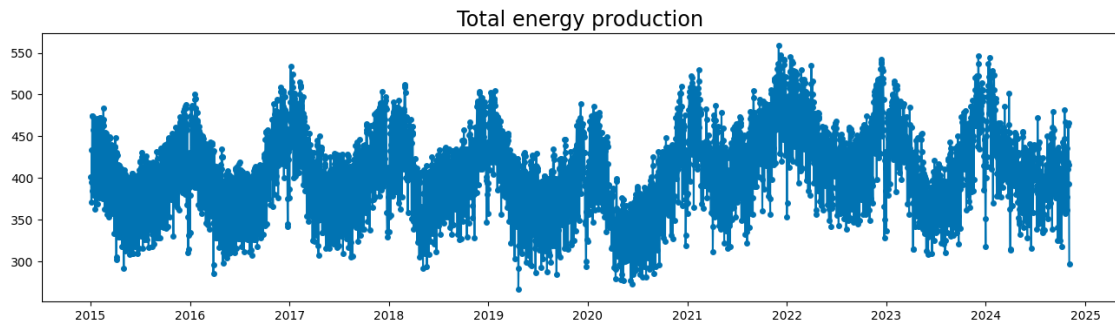
[3594 rows x 6 columns]

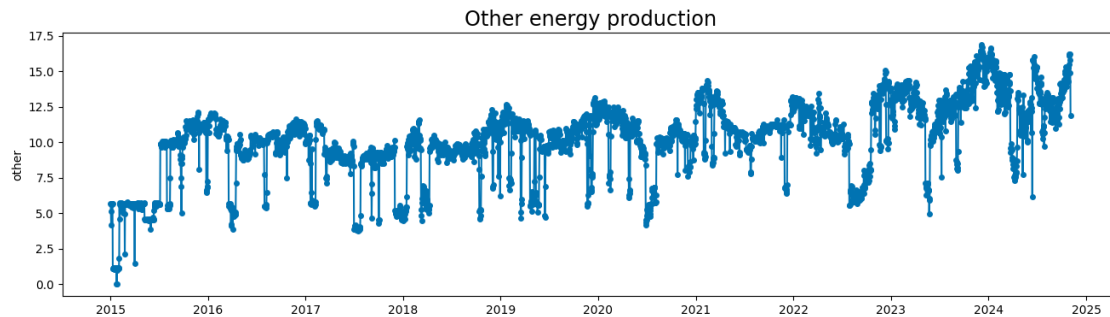
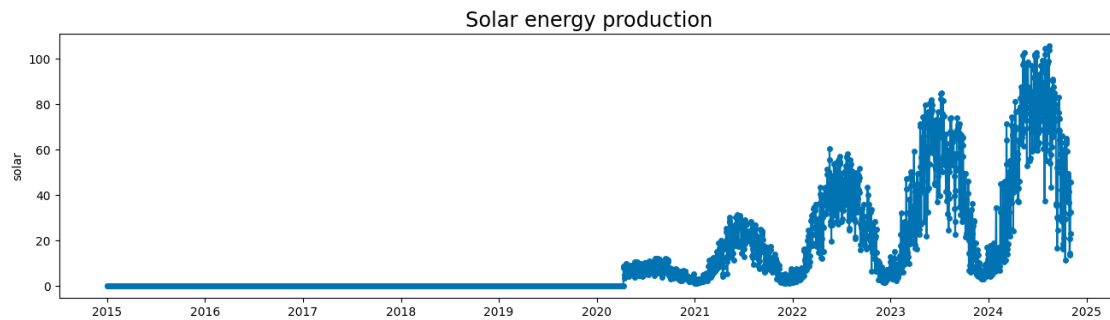
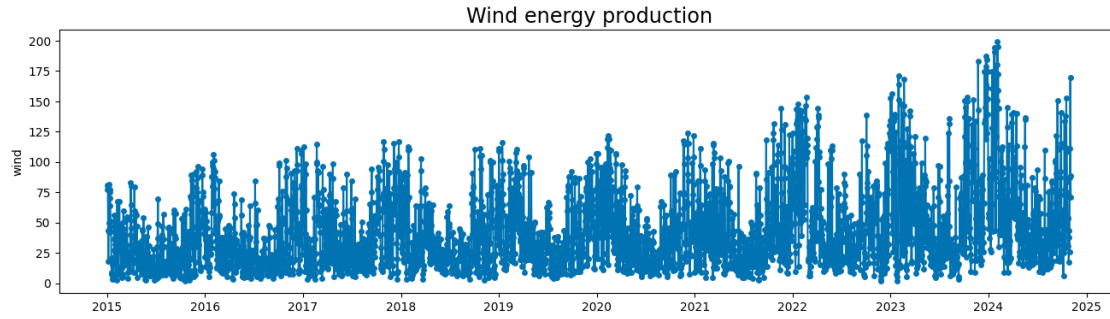
```

[7]: plot_series(df_energy.sum(axis="columns"), title=f"Total energy production")

for col in df_energy.columns:
    plot_series(df_energy[col], title=f"{col.capitalize()} energy production")

```





Evaluation We will perform long-term forecasting, which is a common task on energy production and demand datasets. We will predict production for 2024, using MASE metric.

```
[8]: from sktime.transformations.series.impute import Imputer

df_energy_train = df_energy[df_energy.index < "2024-01-01"]
df_energy_test = df_energy[df_energy.index >= "2024-01-01"]

print(f"Data size: train {len(df_energy_train)}, test {len(df_energy_test)}")
```

Data size: train 3286, test 308

1.2 Forecasting

The sub-sections are independent, and can be implemented in any order. The more you do, the more points (and hence the higher mark) you get. They are also more freeform than previous notebook, and there are more options to choose from.

When tuning hyperparameters, choose any strategy you think is reasonable, taking into consideration computational cost and model complexity. Temporal train-valid-test split, time split CV, expanding window - your choice. Even manual tuning is ok, if you think it makes sense, but remember to use the validation set.

You can use any framework and tool you want, but suggestions are provided in further sections. Install additional dependencies as needed, either using Poetry and recreating `poetry lock`, or by directly using `!pip install ....`

Training and evaluating more models from particular category can get you more points, as described below. If you prefer, you can also experiment with other models, e.g. RNNs, CNN-based, or state-space models (SSMs), adding further sections. Each one by default is worth 2 points.

Warning: when making this notebook, some errors with `neuralforecast` cropped up when horizon was greater than 292 for Italian pasta dataset. You can cut the test set at 292 if necessary.

Note that some frameworks (e.g. `neuralforecast`) require “tall”/“long” time series representation, with columns: `unique_id` (time series identifier), `ds` (date) and `y` (actual value). This is in contrast to the “wide” representation, where we have individual series in separate columns, each row with separate date, and values in cells. See e.g. [neuralforecast quickstart](#) for an example. Functions prepared below may be useful.

```
[9]: from typing import Optional

def wide_to_long_df(df: pd.DataFrame) -> pd.DataFrame:
    df = pd.melt(df, ignore_index=False).reset_index(names="date")
    df = df.rename(columns={"variable": "unique_id", "date": "ds", "value":
↪ "y"})
    return df

def long_to_wide_df(df: pd.DataFrame, values_col: Optional[str] = None) -> pd.
↪ DataFrame:
    if "unique_id" not in df.columns:
        df = df.reset_index(names="unique_id")

    values_col = values_col if values_col else df.columns[-1]
    df = pd.pivot(df, columns="unique_id", index="ds", values=values_col)
    return df
```


1.2.1 Baselines (2 points)

Implement baselines for neural models: - last value (naive) - average - AutoARIMA - AutoETS (with damped trend)

Each dataset is worth 1 point. sktime will be useful.

```
[10]: from sktime.forecasting.naive import NaiveForecaster
from sktime.forecasting.arima import AutoARIMA
from sktime.forecasting.ets import AutoETS
from sktime.performance_metrics.forecasting import mean_absolute_scaled_error
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def evaluate_models(models, train, test, dataset_name):
    results = {}
    print(f"\n--- Evaluating {dataset_name} ---")

    for name, model in models.items():
        print(f"Fitting {name}...")
        try:
            # Fit
            model.fit(train)
            # Predict
            fh = np.arange(1, len(test) + 1)
            y_pred = model.predict(fh)

            # Evaluate (average MASE across columns)
            mase_scores = []
            for col in train.columns:
                pred_col = y_pred[col] if isinstance(y_pred, pd.DataFrame) else y_pred
                mase = mean_absolute_scaled_error(
                    test[col], pred_col, y_train=train[col]
                )
                mase_scores.append(mase)

            avg_mase = np.mean(mase_scores)
            results[name] = avg_mase
            print(f"{name} MASE: {avg_mase:.4f}")

            # Plot first series
            plt.figure(figsize=(10, 4))
            plt.plot(train.index[-100:], train.iloc[-100:, 0], label='Train')
            plt.plot(test.index, test.iloc[:, 0], label='Test')
            plt.plot(test.index, y_pred.iloc[:, 0], label='Pred')
            plt.title(f"{dataset_name} - {name} (First Series)")
```

```

plt.legend()
plt.show()

except Exception as e:
    print(f"Failed to run {name}: {e}")
    results[name] = float('inf')

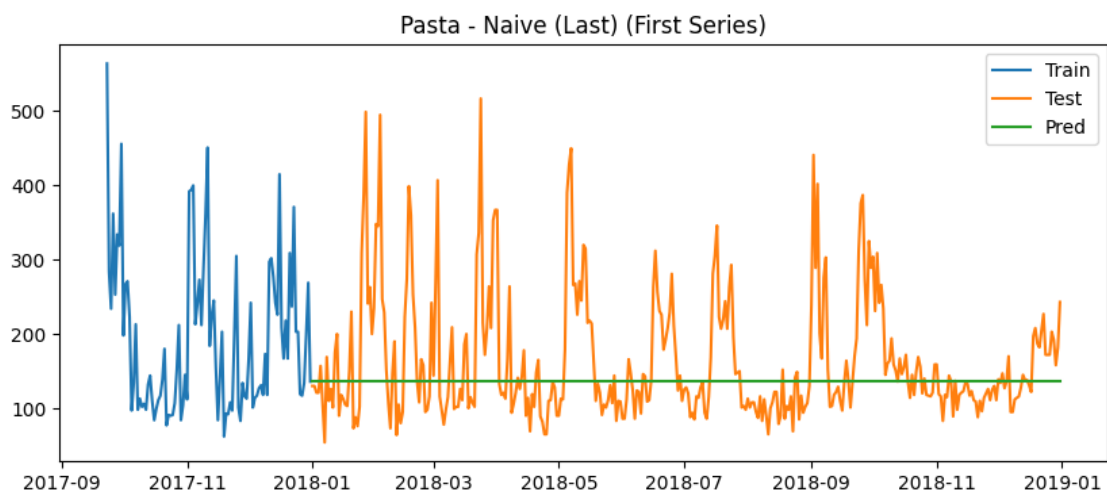
return results

# Pasta Baselines
pasta_models = {
    "Naive (Last)": NaiveForecaster(strategy="last"),
    "Average": NaiveForecaster(strategy="mean"),
    "AutoARIMA": AutoARIMA(sp=7, suppress_warnings=True, stepwise=True,
↳max_p=2, max_q=2),
    "AutoETS": AutoETS(auto=True, sp=7, n_jobs=-1)
}
evaluate_models(pasta_models, df_pasta_train, df_pasta_test, "Pasta")

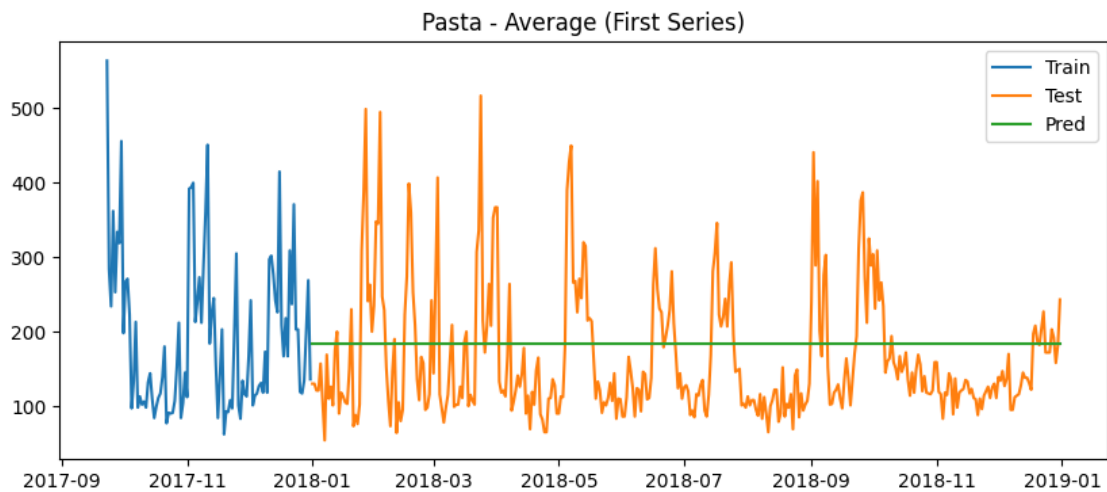
# Energy Baselines
energy_models = {
    "Naive (Last)": NaiveForecaster(strategy="last"),
    "Average": NaiveForecaster(strategy="mean"),
    "AutoETS": AutoETS(auto=True, sp=7, n_jobs=-1)
}
evaluate_models(energy_models, df_energy_train, df_energy_test, "Energy")

```

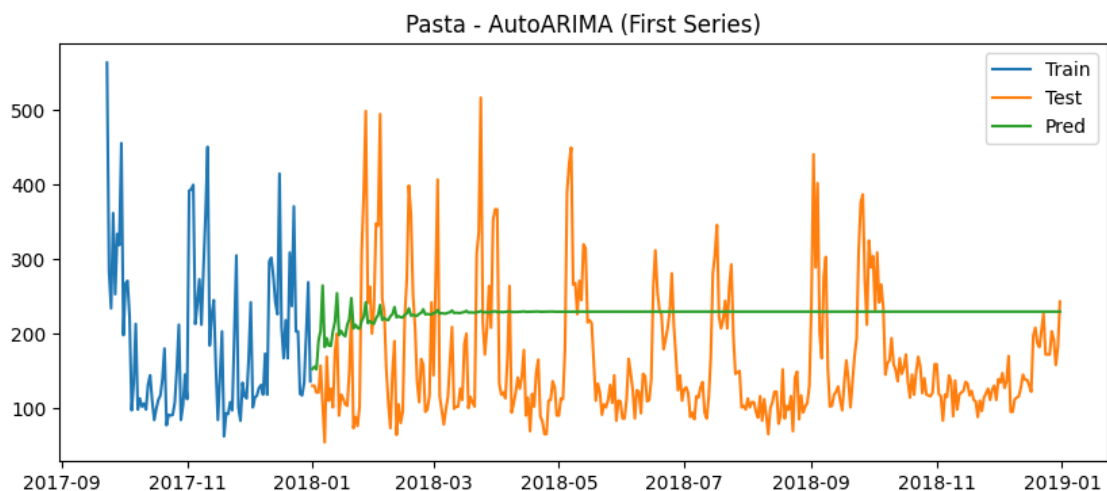
\n--- Evaluating Pasta ---
 Fitting Naive (Last)...
 Naive (Last) MASE: 0.9117



Fitting Average...
Average MASE: 1.2576



Fitting AutoARIMA...
AutoARIMA MASE: 1.2263



Fitting AutoETS...

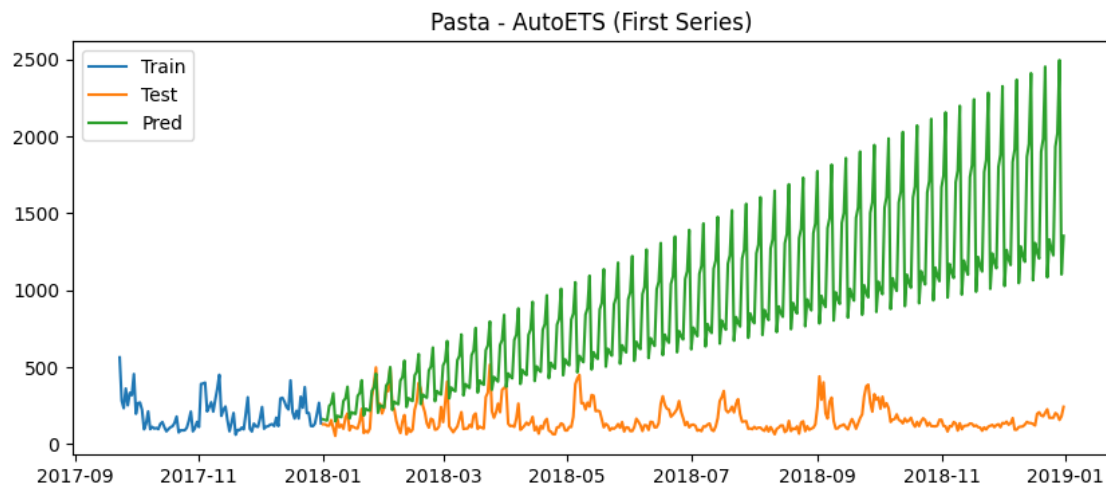
```
/Users/mateu/Education/sem9/ml/ml_time_series_forecasting_course/.venv/lib/pytho  
n3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum  
Likelihood optimization failed to converge. Check mle_retvals  
  warnings.warn("Maximum Likelihood optimization failed to "  
/Users/mateu/Education/sem9/ml/ml_time_series_forecasting_course/.venv/lib/pytho  
n3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum
```

```

Likelihood optimization failed to converge. Check mle_retvals
warnings.warn("Maximum Likelihood optimization failed to "
/Users/mateu/Education/sem9/ml/ml_time_series_forecasting_course/.venv/lib/pytho
n3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum
Likelihood optimization failed to converge. Check mle_retvals
warnings.warn("Maximum Likelihood optimization failed to "

```

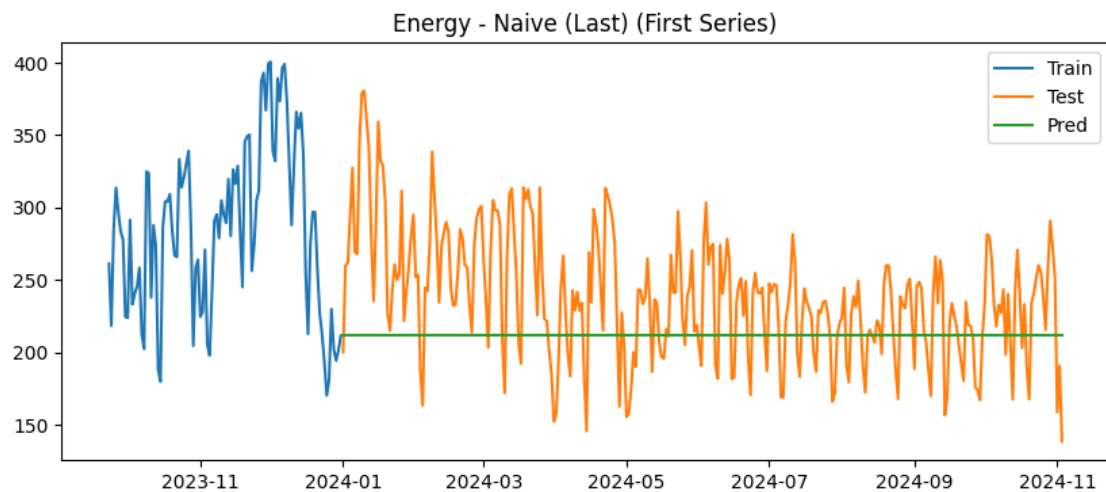
AutoETS MASE: 3.7516



```

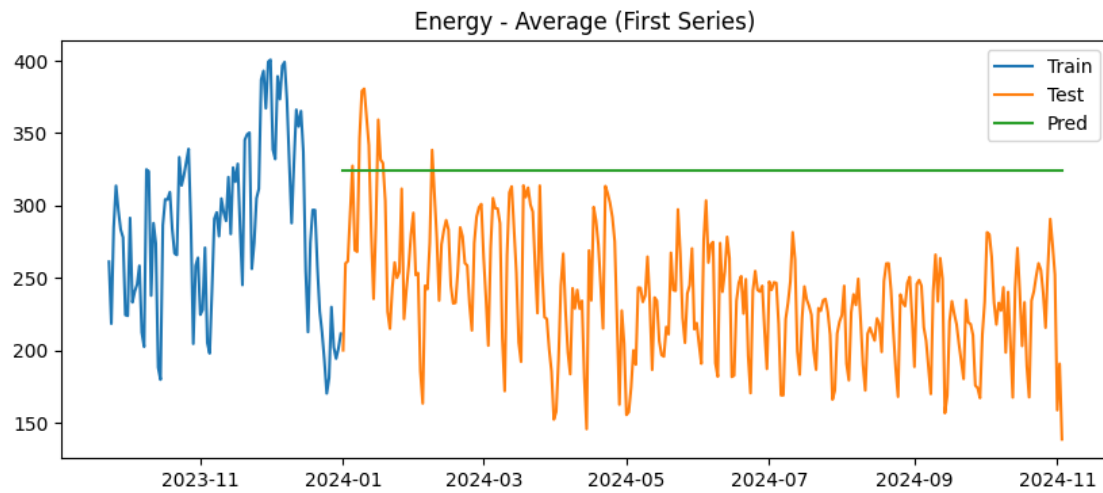
\n--- Evaluating Energy ---
Fitting Naive (Last)...
Naive (Last) MASE: 6.3768

```



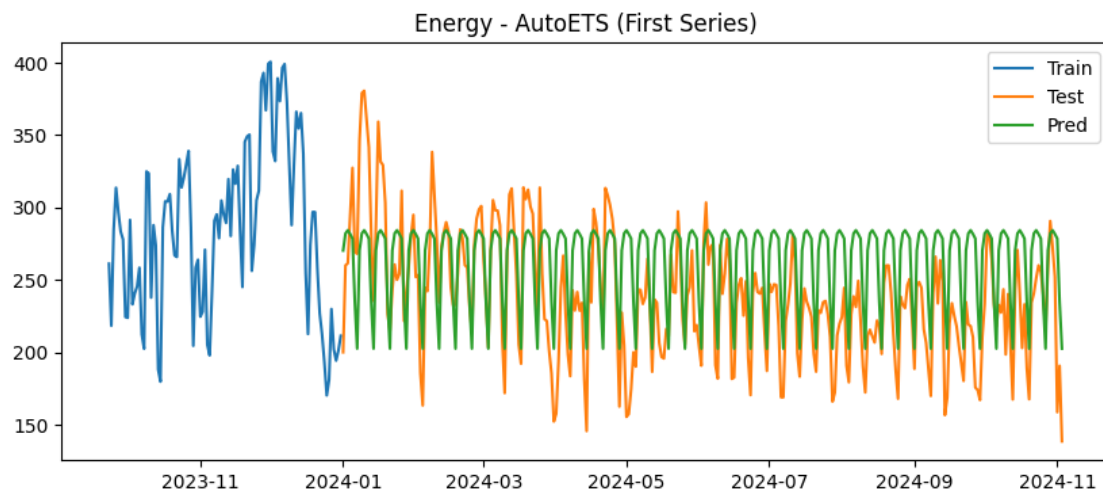
Fitting Average...

Average MASE: 7.8705



Fitting AutoETS...

AutoETS MASE: 9.0909



```
[10]: {'Naive (Last)': 6.376810833859789,  
      'Average': 7.870497780791196,  
      'AutoETS': 9.090913746355353}
```

1.2.2 Linear models (2 points)

Implement linear neural models: - multioutput linear regression - LTSF Linear - LTSF DLinear - LTSF NLinear

Note that Linear is a multi-channel model, while multioutput linear regression is single-channel.

Tune the lookback window, the only hyperparameter of those models, or justify your choice in a comment if you don't. You can check the papers for reasonable values.

If you use a given model, train it on both datasets. Each model is worth 0.5 points. Useful libraries: `sktime`, `neuralforecast`, `PyTorch`.

Useful references: - “Are Transformers Effective for Time Series Forecasting?” A. Zeng et al. (GitHub code) - “An Analysis of Linear Time Series Forecasting Models” W.Toner, L. Darlow (GitHub code)

```
[11]: # Pasta (h=365)
h_pasta = 365
input_size_pasta = 2 * h_pasta

# Energy (h=366)
h_energy = 366
input_size_energy = 2 * h_energy
```

```
[12]: # Convert data to long format for NeuralForecast
df_pasta_long_train = wide_to_long_df(df_pasta_train)
df_pasta_long_test = wide_to_long_df(df_pasta_test)

df_energy_long_train = wide_to_long_df(df_energy_train)
df_energy_long_test = wide_to_long_df(df_energy_test)
```

```
[13]: def train_eval_nf(models, df_train_long, df_test_long, h, freq, dataset_name):
    print(f"\n--- NeuralForecast: {dataset_name} ---")
    # Suppress progress bars to avoid widget errors in headless environments
    for model in models:
        if hasattr(model, 'trainer_kwargs'):
            if model.trainer_kwargs is None:
                model.trainer_kwargs = {}
            model.trainer_kwargs['enable_progress_bar'] = False
            model.trainer_kwargs['logger'] = False
        else:
            try:
                model.trainer_kwargs = {'enable_progress_bar': False, 'logger':
↪ False}
            except Exception:
                pass

    nf = NeuralForecast(models=models, freq=freq)
    # Try passing verbose=False to fit if supported
    try:
        nf.fit(df=df_train_long, verbose=False)
    except TypeError:
        nf.fit(df=df_train_long)
```

```

fcst_df = nf.predict()

fcst_df = fcst_df.reset_index()
test_df = df_test_long.reset_index()
merged = pd.merge(test_df, fcst_df, on=['unique_id', 'ds'], how='inner')

print("Columns in forecast:", fcst_df.columns)

results = {}
for model in models:
    model_name = model.alias if hasattr(model, 'alias') else model.
↪ __class__.__name__

    if model_name in merged.columns:
        mae = (merged['y'] - merged[model_name]).abs().mean()
        results[model_name] = mae
        print(f"{model_name} MAE: {mae:.4f}")
    else:
        print(f"Could not find predictions for {model_name}")

return fcst_df

```

1.2.3 MLP-based models (2 points)

Implement MLP-based neural models: - N-BEATS - TSMixer

For N-BEATS, use the interpretable architecture variant. If you want to tune hyperparameters, you can use e.g. automated class from `neuralforecast` with Ray or Optuna frameworks.

Training each model on each dataset is worth 0.5 points. Useful libraries: `neuralforecast`, PyTorch, `pytorch-tsmixer`.

Other interesting MLP-based models are e.g. N-HiTS, TiDE, TimeMixer, SOFTS. Each additional model is graded like models above.

Useful references: - “N-BEATS: Neural basis expansion analysis for interpretable time series forecasting” B. Oreshkin et al. - “TSMixer: An All-MLP Architecture for Time Series Forecasting” S. Chen et al. - “N-HiTS: Neural Hierarchical Interpolation for Time Series Forecasting” C. Challu et al. - “Long-term Forecasting with TiDE: Time-series Dense Encoder” A. Das et al. - “TimeMixer: Decomposable Multiscale Mixing for Time Series Forecasting” S. Wang et al. - “SOFTS: Efficient Multivariate Time Series Forecasting with Series-Core Fusion” L. Han et al. - `neuralforecast` forecasting models list

```

[14]: from neuralforecast import NeuralForecast
      from neuralforecast.models import NBEATS, TSMixer
      from neuralforecast.losses.pytorch import MAE

      # Pasta

```

```

models_mlp_pasta = [
    NBEATS(h=h_pasta, input_size=input_size_pasta, loss=MAE(), max_steps=100,
    ↪scaler_type='standard'),
    TSMixer(h=h_pasta, input_size=input_size_pasta, loss=MAE(), n_series=4,
    ↪max_steps=100, scaler_type='standard')
]
print("Running MLP Models on Pasta...")
train_eval_nf(models_mlp_pasta, df_pasta_long_train, df_pasta_long_test,
    ↪h_pasta, 'D', "Pasta")

# Energy
models_mlp_energy = [
    NBEATS(h=h_energy, input_size=input_size_energy, loss=MAE(), max_steps=100,
    ↪scaler_type='standard'),
    TSMixer(h=h_energy, input_size=input_size_energy, loss=MAE(), n_series=6,
    ↪max_steps=100, scaler_type='standard')
]
print("Running MLP Models on Energy...")
train_eval_nf(models_mlp_energy, df_energy_long_train, df_energy_long_test,
    ↪h_energy, 'D', "Energy")

```

Seed set to 1

Seed set to 1

Running MLP Models on Pasta...

--- NeuralForecast: Pasta ---

GPU available: True (mps), used: True

TPU available: False, using: 0 TPU cores

	Name	Type	Params	Mode	FLOPs
0	loss	MAE	0	train	0
1	padder_train	ConstantPad1d	0	train	0
2	scaler	TemporalNorm	0	train	0
3	blocks	ModuleList	5.6 M	train	0

Trainable params: 4.8 M

Non-trainable params: 800 K

Total params: 5.6 M

Total estimated model params size (MB): 22

Modules in train mode: 31

Modules in eval mode: 0

Total FLOPs: 0


```

`Trainer.fit` stopped: `max_steps=100` reached.
GPU available: True (mps), used: True
TPU available: False, using: 0 TPU cores

```

	Name	Type	Params	Mode	FLOPs
0	loss	MAE	0	train	0
1	padder_train	ConstantPad1d	0	train	0
2	scaler	TemporalNorm	0	train	0
3	norm	RevINMultivariate	8	train	0
4	mixing_layers	Sequential	1.1 M	train	0
5	out	Linear	266 K	train	0

```

Trainable params: 1.4 M
Non-trainable params: 0
Total params: 1.4 M
Total estimated model params size (MB): 5
Modules in train mode: 28
Modules in eval mode: 0
Total FLOPs: 0

```

```

`Trainer.fit` stopped: `max_steps=100` reached.
GPU available: True (mps), used: True
TPU available: False, using: 0 TPU cores
GPU available: True (mps), used: True
TPU available: False, using: 0 TPU cores
Seed set to 1
Seed set to 1
GPU available: True (mps), used: True
TPU available: False, using: 0 TPU cores

```

```

Columns in forecast: Index(['index', 'unique_id', 'ds', 'NBEATS', 'TSMixer'],
dtype='object')
Could not find predictions for None
Could not find predictions for None
Running MLP Models on Energy..

```

```

--- NeuralForecast: Energy ---

```

	Name	Type	Params	Mode	FLOPs
0	loss	MAE	0	train	0
1	padder_train	ConstantPad1d	0	train	0
2	scaler	TemporalNorm	0	train	0
3	blocks	ModuleList	5.6 M	train	0

Trainable params: 4.8 M
 Non-trainable params: 804 K
 Total params: 5.6 M
 Total estimated model params size (MB): 22
 Modules in train mode: 31
 Modules in eval mode: 0
 Total FLOPs: 0

`Trainer.fit` stopped: `max\_steps=100` reached.
 GPU available: True (mps), used: True
 TPU available: False, using: 0 TPU cores

	Name	Type	Params	Mode	FLOPs
0	loss	MAE	0	train	0
1	padder_train	ConstantPad1d	0	train	0
2	scaler	TemporalNorm	0	train	0
3	norm	RevINMultivariate	12	train	0
4	mixing_layers	Sequential	1.1 M	train	0
5	out	Linear	268 K	train	0

Trainable params: 1.4 M
 Non-trainable params: 0
 Total params: 1.4 M
 Total estimated model params size (MB): 5
 Modules in train mode: 28
 Modules in eval mode: 0
 Total FLOPs: 0

`Trainer.fit` stopped: `max\_steps=100` reached.
 GPU available: True (mps), used: True
 TPU available: False, using: 0 TPU cores
 GPU available: True (mps), used: True
 TPU available: False, using: 0 TPU cores

Columns in forecast: Index(['index', 'unique_id', 'ds', 'NBEATS', 'TSMixer'],
 dtype='object')
 Could not find predictions for None
 Could not find predictions for None

```
[14]:      index unique_id      ds      NBEATS      TSMixer
      0          0    coal 2024-01-01  293.922607  278.597443
```

1	1	coal	2024-01-02	281.527039	297.813416
2	2	coal	2024-01-03	304.729645	306.604675
3	3	coal	2024-01-04	307.159729	307.939636
4	4	coal	2024-01-05	315.286469	287.399689
...	...	...	...	...	...
2191	2191	wind	2024-12-27	110.004547	122.155655
2192	2192	wind	2024-12-28	99.960762	106.244835
2193	2193	wind	2024-12-29	109.487701	119.083847
2194	2194	wind	2024-12-30	106.243515	117.075806
2195	2195	wind	2024-12-31	110.640366	113.619537

[2196 rows x 5 columns]

1.2.4 Time series transformers (2 points)

Implement a time series transformer, e.g. PatchTST.

You can use either pretrained variant, or train from scratch. If you want to tune hyperparameters, you can use e.g. automated class from **neuralforecast** with Ray or Optuna frameworks.

Training the model in any way is worth 2 points. You can also choose any other time series transformer, e.g. TFT, iTransformer, Autoformer. Useful libraries: **neuralforecast**, PyTorch, **transformers**, IBM Granite. Each model after the first one is worth 1 point. If you use PatchTST, using the pretrained one and training from scratch counts as two separate models.

Useful references: - [“A Time Series is Worth 64 Words: Long-term Forecasting with Transformers” Y. Nie et al.](#) - [“Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting” B. Lim et al.](#) - [“iTransformer: Inverted Transformers Are Effective for Time Series Forecasting” Y. Liu et al.](#) - [“Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting” H. Wu et al.](#) - [neuralforecast forecasting models list](#) - [IBM Granite tutorial for pretrained PatchTST](#)

```
[15]: from neuralforecast import NeuralForecast

from neuralforecast.models import PatchTST

models_trans_pasta = [
    PatchTST(h=h_pasta, input_size=input_size_pasta, loss=MAE(), max_steps=100,
    ↪scaler_type='standard', patch_len=24, stride=12)
]
print("Running Transformer on Pasta...")
train_eval_nf(models_trans_pasta, df_pasta_long_train, df_pasta_long_test,
    ↪h_pasta, 'D', "Pasta")

models_trans_energy = [
    PatchTST(h=h_energy, input_size=input_size_energy, loss=MAE(),
    ↪max_steps=100, scaler_type='standard', patch_len=24, stride=12)
]
print("Running Transformer on Energy...")
```

```
train_eval_nf(models_trans_energy, df_energy_long_train, df_energy_long_test,
↳h_energy, 'D', "Energy")
```

Seed set to 1

GPU available: True (mps), used: True

TPU available: False, using: 0 TPU cores

Running Transformer on Pasta...

--- NeuralForecast: Pasta ---

	Name	Type	Params	Mode	FLOPs
0	loss	MAE	0	train	0
1	padder_train	ConstantPad1d	0	train	0
2	scaler	TemporalNorm	0	train	0
3	model	PatchTST_backbone	3.2 M	train	0

Trainable params: 3.2 M

Non-trainable params: 3

Total params: 3.2 M

Total estimated model params size (MB): 12

Modules in train mode: 90

Modules in eval mode: 0

Total FLOPs: 0

`Trainer.fit` stopped: `max\_steps=100` reached.

GPU available: True (mps), used: True

TPU available: False, using: 0 TPU cores

Seed set to 1

GPU available: True (mps), used: True

TPU available: False, using: 0 TPU cores

Columns in forecast: Index(['index', 'unique_id', 'ds', 'PatchTST'],
dtype='object')

Could not find predictions for None

Running Transformer on Energy...

--- NeuralForecast: Energy ---

	Name	Type	Params	Mode	FLOPs
0	loss	MAE	0	train	0
1	padder_train	ConstantPad1d	0	train	0
2	scaler	TemporalNorm	0	train	0
3	model	PatchTST_backbone	3.3 M	train	0

```

Trainable params: 3.3 M
Non-trainable params: 3
Total params: 3.3 M
Total estimated model params size (MB): 13
Modules in train mode: 90
Modules in eval mode: 0
Total FLOPs: 0

```

```

`Trainer.fit` stopped: `max_steps=100` reached.
GPU available: True (mps), used: True
TPU available: False, using: 0 TPU cores

```

```

Columns in forecast: Index(['index', 'unique_id', 'ds', 'PatchTST'],
dtype='object')

```

```

Could not find predictions for None

```

```

[15]:      index unique_id      ds    PatchTST
0         0      coal 2024-01-01  275.657745
1         1      coal 2024-01-02  292.746643
2         2      coal 2024-01-03  307.319702
3         3      coal 2024-01-04  289.821350
4         4      coal 2024-01-05  285.402802
...
2191    2191      wind 2024-12-27  104.628860
2192    2192      wind 2024-12-28  109.092468
2193    2193      wind 2024-12-29   96.463181
2194    2194      wind 2024-12-30   90.474976
2195    2195      wind 2024-12-31   93.640205

```

```

[2196 rows x 4 columns]

```

1.2.5 Pretrained foundation models (2 points)

Use a pretrained time series foundation model for zero-shot forecasting.

Examples are e.g. TimesFM, Lag-Llama, TimeGPT, Moirai. Model notes: 1. TimesFM - using the PyTorch version of original library is suggested 2. Lag-Llama - this is a probabilistic model, note that we are interested in point forecasts (mean probabilistic value) 3. TimeGPT - as this is a proprietary model, you need to provide the API token, make sure you don't push it to a public repository!

The first model is worth 2 points, and subsequent ones are worth 1 point each.

Useful references: - [“A decoder-only foundation model for time-series forecasting” A. Das et al.](#)
- [TimesFM repository](#) - [“Lag-Llama: Towards Foundation Models for Probabilistic Time Series Forecasting” K. Rasul et al.](#) - [Lag-Llama repository](#) - [“TimeGPT-1” A. Garza et al.](#) - [TimeGPT](#)

docs - “Unified Training of Universal Time Series Forecasting Transformers” G. Woo et al. -
HuggingFace Moirai model page

```
[16]: import timesfm

# Initialize TimesFM model
# Using google/timesfm-1.0-200m-pytorch
# We will use defaults for h usually, but constructor takes h.
tfm = timesfm.TimesFm(
    hparams=timesfm.TimesFmHparams(backend='cpu', horizon_len=h_pasta),
    checkpoint=timesfm.TimesFmCheckpoint(huggingface_repo_id="google/timesfm-1.0-200m-pytorch"),
)

print("Running TimesFM on Pasta...")
tfm_forecast_pasta = tfm.forecast_on_df(
    inputs=df_pasta_long_train,
    freq="D",
    value_name="y",
    model_name="TimesFM",
)
print("TimesFM Pasta done.")

# Evaluate Pasta
tfm_pasta_eval = pd.merge(df_pasta_long_test.reset_index(), tfm_forecast_pasta,
    on=['unique_id', 'ds'], how='inner')
mae_pasta = (tfm_pasta_eval['y'] - tfm_pasta_eval['TimesFM']).abs().mean()
print(f"TimesFM Pasta MAE: {mae_pasta:.4f}")

# Energy
tfm_energy = timesfm.TimesFm(
    hparams=timesfm.TimesFmHparams(backend='cpu', horizon_len=h_energy),
    checkpoint=timesfm.TimesFmCheckpoint(huggingface_repo_id="google/timesfm-1.0-200m-pytorch"),
)

print("Running TimesFM on Energy...")
tfm_forecast_energy = tfm_energy.forecast_on_df(
    inputs=df_energy_long_train,
    freq="D",
    value_name="y",
    model_name="TimesFM",
)
print("TimesFM Energy done.")

# Evaluate Energy
```

```
tfm_energy_eval = pd.merge(df_energy_long_test.reset_index(),  
    ↪tfm_forecast_energy, on=['unique_id', 'ds'], how='inner')  
mae_energy = (tfm_energy_eval['y'] - tfm_energy_eval['TimesFM']).abs().mean()  
print(f"TimesFM Energy MAE: {mae_energy:.4f}")
```

See <https://github.com/google-research/timesfm/blob/master/README.md> for updated APIs.

Loaded PyTorch TimesFM, likely because python version is 3.12.8 (main, Jan 14 2025, 23:36:58) [Clang 19.1.6].

Fetching 3 files: 0%| | 0/3 [00:00<?, ?it/s]

Running TimesFM on Pasta...

Processing dataframe with single process.

Finished preprocessing dataframe.

Finished forecasting.

TimesFM Pasta done.

TimesFM Pasta MAE: 38.5657

Fetching 3 files: 0%| | 0/3 [00:00<?, ?it/s]

Running TimesFM on Energy...

Processing dataframe with single process.

Finished preprocessing dataframe.

Finished forecasting.

TimesFM Energy done.

TimesFM Energy MAE: 18.6358